Technical Specification Document

Zudobot — AI Chatbot SaaS Platform

Field	Value
---	---
Document name	Zudobot Technical Specification Document
File	docs/TSD-Zudobot-20260705-v1.1.md
Version	v1.1
Date	2026-07-05
Status	Source-code aligned technical specification
Owner	Zudogu Engineering
Audience	Engineers, DevOps, QA, Product, Solution Architect

---

1. Purpose and Scope

เอกสารนี้อธิบายสเปคทางเทคนิคของระบบ Zudobot เพื่อใช้เป็นแนวทางพัฒนาและดำเนินการในระยะต่อไป โดยครอบคลุม:

- Overview architecture
- Data diagram and ER diagram
- System flow and business flow by process
- Data dictionary
- API flow and API specification
- Source code structure
- Error handling and message mapping
- Function / feature / middleware / command mapping
- Cloud, database, framework, AI, and runtime specifications

---

2. Technology and Environment Specification

2.1 Platform and Cloud

- Cloud platform: AWS Amplify
- Region: ap-southeast-2 (Sydney)
- Deployment model: Mono-repo multi-app deployment
- apps/web
- apps/api
- apps/dashboard
- apps/landing
- packages/widget
- Runtime: Node.js + Next.js serverless/SSR runtime on Amplify
- Environment variables: must come from AWS Amplify Console only; no app-local .env runtime usage

2.2 Database

- Database: MongoDB Atlas
- Database name: zudobot_saas
- Driver: Mongoose
- Main collections/models:
- User
- TenantProfile
- Subscription
- ChatSession
- ConversationSession
- KnowledgeChunk
- KnowledgeJob
- KnowledgeRefreshSchedule
- Product
- OrderDraft / Order / OrderOutbox
- PartnerProfile / PartnerInvoice / PartnerClientData
- VipTenant
- Notification / SecurityState / ZudobotConfig

2.3 Framework and Languages

- Frontend: Next.js 14, React 18, Tailwind CSS, Radix UI, Lucide
- Backend: Next.js App Router, Route Handlers, TypeScript
- Widget: Vite + TypeScript (IIFE bundle)
- Authentication: NextAuth v5 beta with Google OAuth
- Validation: Zod
- AI SDK: @google/generative-ai
- Payment: Stripe
- Email: Resend

- Misc: pdf-parse, mammoth, xlsx, docx

2.4 AI Specification

- Primary AI provider: Google Gemini
- Models used in current implementation:
- text-embedding-004 for vector knowledge embedding
- Gemini chat model for response generation and self-learning
- Safety/error handling: quota exhausted, auth failure, invalid history, service unavailable are normalized to user-safe messages

2.5 Server and Runtime Spec

- Web app port: 3000
- API app port: 4000
- Dashboard app port: 4001
- Node.js version: 20+ recommended
- Package manager: npm workspaces

---

3. Overview Architecture

```
Client Layer
  - Website / Admin / Tenant / Partner / LINE / Meta / TikTok
      |
      v
AWS Amplify (ap-southeast-2)
  - apps/web : auth, dashboard, admin, public APIs, widget APIs
  - apps/api : chatbot API
  - apps/dashboard : standalone analytics/dashboard
  - apps/landing : public landing page
  - packages/widget : embeddable widget bundle
      |
      +--> MongoDB Atlas (zudobot_saas)
      +--> Gemini AI (chat + embeddings)
      +--> Stripe (checkout, subscription, webhook)
      +--> LINE / Meta / TikTok webhooks
      +--> Resend / Redis / OAuth providers
```

3.1 Core Responsibilities

- Presentation layer: landing page, admin UI, tenant dashboard, partner portal
- Authentication and authorization: Google OAuth, JWT, middleware-based route guards
- Business logic: billing, widget runtime, knowledge ingestion, order flow, omnichannel handoff

- Data access: Mongoose models and MongoDB collections
- Integrations: Gemini AI, Stripe, webhooks, email, rate limiting

---

4. Data Diagram

```
User Sign-in
  -> NextAuth Google OAuth
  -> JWT token with role/tenant/onboarding info
  -> Middleware authorization
  -> Dashboard/Admin/Partner route access

Widget Chat Request
  -> /api/widget/init validates embed key and origin
  -> /api/widget/chat validates quota/domain/session
  -> retrieve tenant context + knowledge chunks
  -> call Gemini AI
  -> persist session and usage data
  -> return reply / products / handoff

Knowledge Ingestion
  -> upload/URL/custom text
  -> create KnowledgeJob
  -> chunk/parse/embed text
  -> store KnowledgeChunk + refresh schedule
```

4.1 Data Exchange Summary

Process	Input	Storage/Service	Output
---	---	---	---
Authentication	Google auth profile	User + JWT	Auth session + onboarding state
Widget init	embedKey + origin	TenantProfile	Widget config
Widget chat	message + sessionId	ConversationSession + Gemini + KnowledgeChunk	AI reply + usage log
Knowledge ingest	file / URL / text	KnowledgeJob + KnowledgeChunk	Knowledge ready for RAG
Billing	checkout/webhook	Subscription + Invoice	Plan status
Omnichannel	webhook event	ConversationSession + Notification	Inbox / handoff alert
Partner	invite/provision	PartnerProfile + Subscription	Partner-managed tenant access

---

5. System Flow by Process

5.1 Authentication and Onboarding Flow

1. User accesses login page.
2. Google OAuth sign-in completes.
3. System creates or reuses user identity.
4. JWT carries role, tenant state, pending registration information.
5. Middleware routes user to onboarding, dashboard, admin, or partner portal.

5.2 Widget Initialization Flow

1. Website loads widget script.
2. Browser calls /api/widget/init.
3. System validates embed key and domain whitelist.
4. Widget config is returned to client.

5.3 Widget Messaging Flow

1. Customer sends a message from embedded widget.
2. API validates rate limit, tenant domain, quota, and session state.
3. System detects buying intent / handoff intent.
4. Knowledge chunks and product context are fetched.
5. Gemini generates reply.
6. Session, usage, order draft, and handoff events are persisted.

5.4 Knowledge Management Flow

1. Tenant uploads content or provides URL/text.
2. System creates a knowledge processing job.
3. Content is parsed, chunked, embedded, and saved.
4. Refresh/re-embed schedule is stored for later updates.

5.5 Billing and Subscription Flow

1. User selects a plan or package.
2. Checkout is initiated through Stripe.
3. Stripe webhook updates subscription and invoice status.
4. Bot state machine recalculates plan status and quota access.

5.6 Omnichannel and Webhook Flow

1. External webhook from LINE/Meta/TikTok arrives.
2. System validates token/signature and tenant identity.
3. Conversation session or notification state is updated.
4. Handoff or alert is triggered.

5.7 Cron and Maintenance Flow

1. Scheduled cron endpoint is called with internal secret.
2. Relevant background jobs run such as refresh, backup, purge, sync, and billing operations.
3. Results are persisted and monitoring logs are updated.

---

6. Business Flow by Process

6.1 New Tenant Onboarding

1. Tenant signs in with Google.
2. System marks onboarding as pending.
3. Tenant completes business profile and bot configuration.
4. Tenant receives embed key and widget configuration.
5. Widget is enabled and ready for chat.

6.2 Customer Support and Sales Flow

1. Customer interacts with widget.

2. Bot answers general questions.

3. Bot can recommend products or initiate checkout.

4. If needed, it transfers to a human handoff flow.

5. Order or follow-up data is stored for future operations.

6.3 Knowledge Expansion Flow

1. Admin uploads knowledge base.

2. System processes and indexes it.

3. Bot uses the indexed content for future answers.

4. Refresh schedules keep knowledge up to date.

6.4 Partner Provisioning Flow

1. Partner joins via invite or public join route.

2. KYC/provisioning data is captured.

3. Partner is granted access to client and billing data.

4. Partner-managed tenant access is enabled.

---

7. ER Diagram

```
User 1---1 TenantProfile
User 1---* Subscription
User 1---* Notification
TenantProfile 1---* ConversationSession
TenantProfile 1---* KnowledgeChunk
TenantProfile 1---* Product
TenantProfile 1---* OrderDraft
TenantProfile 1---* ChannelContextToken
TenantProfile 1---* VipTenant
PartnerProfile 1---* Subscription
PartnerProfile 1---* PartnerClientData
PartnerProfile 1---* PartnerInvoice
```

7.1 Main Relationships

- User owns tenant identity and auth state
- TenantProfile configures bot, widget, domain access, and quotas
- Subscription determines billing and bot state
- ConversationSession and ChatSession store user conversations
- KnowledgeChunk and KnowledgeJob drive the retrieval layer
- OrderDraft/OrderOutbox support commerce handoff flows
- PartnerProfile aggregates partner-related subscriptions and invoices

---

8. Data Dictionary

Model / Collection	Purpose	Key Fields
---	---	---
User	Identity and role	email, role, roles, tenantId, onboardingComplete, twoFactorEnabled
TenantProfile	Tenant configuration	embedKey, businessName, widgetEnabled, allowedDomain, allowedDomains, dailyMessageCount
Subscription	Billing state	plan, status, stripeCustomerId, currentPeriodEnd
ChatSession	Conversation history	tenantId, sessionId, messages
ConversationSession	Live/widget session state	tenantId, sessionId, botStatus, handoffRequested
KnowledgeChunk	Indexed vector content	tenantId, text, embedding, source
KnowledgeJob	Batch ingestion state	tenantId, status, progress, error
KnowledgeRefreshSchedule	Auto-refresh config	tenantId, schedule, lastRun
Product	Product catalog	tenantId, name, price, slug
OrderDraft / Order / OrderOutbox	Order flow	status, orderPayload, relayState
PartnerProfile	Partner account	email, userId, legalProfile
VipTenant	VIP override rules	email, tenantId, active
Notification	Platform alert record	type, tenantId, isRead

---

9. API Flow by Process

9.1 Authentication APIs

Flow	Route	Purpose
---	---	---
Google auth	/api/auth/[...nextauth]	Login and session handling
Sync registration	/api/auth/sync-registration	Completes pending registration
2FA	/api/tenant/2fa/*	Setup and verify TOTP
Account delete	/api/tenant/account/*	Soft-delete and recovery
Admin delete	/api/admin/delete-account	Super-admin cascaded deletion

9.2 Widget APIs

Flow	Route	Purpose
---	---	---
Widget init	/api/widget/init	Validate key and origin
Widget chat	/api/widget/chat	Main AI chat route
Widget upload	/api/widget/upload	File upload for widget context
Widget memory	/api/widget/customer-memory	Memory and visitor context
Widget checkout	/api/widget/checkout	Checkout handoff

9.3 Tenant / Admin / Partner APIs

Flow	Route	Purpose
---	---	---
Tenant profile	/api/tenant/me, /api/tenant/store-config	Tenant setup
Knowledge	/api/tenant/knowledge/*	CRUD and processing
Products	/api/tenant/products	Catalog management
Orders	/api/tenant/orders/*	Order listing and slips
Live chat	/api/tenant/live-chat/*	Human handoff workflow
Channels	/api/tenant/channels	Webhook/channel setup
Partner	/api/partner/*	Partner provisioning and analytics
Admin	/api/admin/*	Management and reporting
Stripe	/api/stripe/*	Checkout and webhook processing
Cron	/api/cron/*	Maintenance and background jobs

10. API Specification (Key Routes)

10.1 Public Widget Init

- Method: POST
- Path: /api/widget/init
- Body: { "key": "<embedKey>" }
- Response: { ok, config }
- Errors: missing_key, missing_origin, invalid_key, widget_disabled, domain_not_allowed

10.2 Widget Chat

- Method: POST
- Path: /api/widget/chat
- Body: { key, sessionId, message, attachments?, consentGiven? }
- Response: { ok, reply, blocked?, handoffMode?, products? }
- Security: rate limiting, domain allow-list, quota gate, PII scrubbing, usage increment

10.3 Tenant Knowledge Upload

- Method: POST
- Path: /api/tenant/knowledge/upload
- Purpose: upload document or text for ingestion
- Response: job status or upload ack

10.4 Stripe Checkout

- Method: POST
- Path: /api/stripe/checkout
- Purpose: create subscription/checkout session
- Webhook: /api/stripe/webhook

10.5 Omnichannel Webhook

- Method: POST
- Path: /api/webhooks/line/[tenantId], /api/webhooks/meta/[tenantId], /api/webhooks/tiktok/[tenantId]
- Purpose: inbound message events and handoff integration

---

11. Source Code Specification

Area	Path	Responsibility
---	---	---
Web app	apps/web	Next.js app containing UI, APIs, auth, admin, tenant, partner, widget routes
API app	apps/api	Dedicated API runtime for chatbot-related operations
Dashboard	apps/dashboard	Standalone dashboard app
Widget package	packages/widget	Embeddable widget JS bundle
Middleware	apps/web/middleware.ts	Route guard, role-based access, onboarding restrictions
Env guard	apps/web/lib/env/amplifyGuardrail.ts	Enforces Amplify env variables and prohibits fallback secrets
Models	apps/web/lib/db/models	MongoDB schemas and Mongoose models
AI logic	apps/web/lib/ai	Gemini integration, embedding, error handling, self-learning
Widget logic	apps/web/lib/widget	Quota gate, domain validation, platform-site access
Knowledge logic	apps/web/lib/knowledge	Vector search, RAG event logging
Orders	apps/web/lib/orders	Draft extraction, slip handling, relay
Services	apps/web/lib/services	LINE notify, outbound integrations
Scripts	scripts	Seed, migration, maintenance, env helpers

12. Error Handling Strategy

12.1 Global Error Handling Principles

- User-facing errors are normalized and safe.
- Internal stack details are not exposed to clients.
- High-risk failures are logged server-side.
- Authentication and missing env vars fail fast.

12.2 Common Error Categories

Category	Example	Handling
---	---	---
Validation error	missing_fields, invalid_body	Return 400 with friendly message
Authorization error	invalid_key, domain_not_allowed, widget_disabled	Return 403
Quota/rate limit	rate_limited, blocked	Return 429 or safe blocked response

AI failure	gemini_quota_exhausted, gemini_service_unavailable	Return user-safe retry message
DB failure	database connection error	Log and return 500-safe response
Webhook signature failure	invalid signature	Reject request and log

13. Error Message Mapping

Code	HTTP	Meaning	User Message
missing_fields	400	Required body fields missing	กรุณากรอกข้อมูลให้ครบถ้วน
invalid_body	400	JSON body invalid	ข้อมูลส่งมาไม่ถูกต้อง
message_too_long	400	Message exceeds length limit	ข้อความยาวเกินกว่าที่ระบบรองรับ
missing_origin	403	Origin/header missing	ไม่พบแหล่งที่มาในการเรียกใช้
invalid_key	403	Embed key invalid	คีย์สำหรับ Widget ไม่ถูกต้อง
widget_disabled	403	Widget disabled for tenant	Widget ถูกปิดใช้งานชั่วคราว
domain_not_allowed	403	Domain not allowed	โดเมนนี้ไม่ได้รับอนุญาตให้ใช้งาน
rate_limited	429	Too many requests	กรุณารอสักครู่แล้วลองใหม่
gemini_quota_exhausted	503	AI quota exhausted	ระบบ AI เต็มชั่วคราว กรุณารอ 1 นาที
gemini_service_unavailable	503	AI service unavailable	ระบบ AI ขัดข้องชั่วคราว
gemini_auth_failed	503	Invalid API key	การเชื่อมต่อ AI ล้มเหลว

14. Warning Message Mapping

Warning Type	Trigger	Message
quota_80	80% monthly quota reached	โควต้าใช้เกิน 80% แล้ว
quota_95	95% monthly quota reached	โควต้าใกล้เต็ม กรุณาตรวจสอบแผน
pending_delete	account soft-delete pending	บัญชีกำลังถูกลบและสามารถกู้คืนได้ก่อนกำหนด
pending_registration	new user onboarding pending	กรุณาเสร็จสิ้นการลงทะเบียนก่อนใช้งาน
2fa_pending	2FA verification pending	กรุณายืนยันรหัส 2FA เพื่อใช้งานต่อ
webhook_retry	webhook processing retry	ระบบกำลัง retry webhook

15. Function / Feature / Middleware / Command Mapping

Category	Implementation	Purpose
Route guard	apps/web/middleware.ts	Protects pages and APIs based on auth/role/onboarding state
Env guardrail	apps/web/lib/env/amplifyGuardrail.ts	Enforces Amplify environment variable policy
Widget auth	apps/web/app/api/widget/init/route.ts	Validates widget embed access
Widget chat	apps/web/app/api/widget/chat/route.ts	Main AI conversation flow
Knowledge pipeline	apps/web/app/api/tenant/knowledge/*	Knowledge ingestion and processing
Billing	apps/web/app/api/stripe/*	Subscription checkout and webhook
Omnichannel	apps/web/app/api/webhooks/*	LINE/Meta/TikTok inbound handling
Partner flow	apps/web/app/api/partner/*	Invite, verify, provision, billing
Admin operations	apps/web/app/api/admin/*	Tenant and system control
Cron jobs	apps/web/app/api/cron/*	Maintenance and scheduled operations
Build commands	package.json	Root build/dev scripts
App commands	apps/web/package.json, apps/api/package.json, apps/dashboard/package.json, packages/widget/package.json	Local dev/build for each app

15.1 Key Development Commands

Command	Purpose
npm run dev	Run web app locally
npm run dev:api	Run API app locally
npm run dev:dashboard	Run dashboard app locally
npm run build:widget	Build widget bundle
npm run build	Build web app
npm run build:api	Build API app
npm run build:dashboard	Build dashboard app
npm run docker:up	Start local Docker stack

npm run db:test-mongo	Validate MongoDB connection
npm run security:gate	Run security gate checks

---

16. Security and Operational Notes

- Secrets and config must come from AWS Amplify Console.
- No runtime fallback strings for critical secrets.
- Google OAuth only; no email/password or magic-link authentication.
- Role-based access supported: super_admin, admin, tenant, partner_admin.
- Account deletion uses protected admin route with dry-run and cascade capability.
- Middleware enforces onboarding, 2FA, deletion, and role-specific routing.

---

17. Additional Technical Details

17.1 Bot State Machine

ระบบกำหนดสถานะบอทตามแผนและโควต้า โดยใช้ลำดับต่อไปนี้:

- trial
- trial_quota_daily_exhausted
- trial_expired
- active
- grace_5pct
- suspended_quota
- suspended_payment
- pending_kyc
- disabled

การคำนวณสถานะทำผ่าน pure function evaluateBotState **ใน** apps/web/lib/payment/botStateMachine.ts **โดยไม่พึ่ง** DB **และส่งกลับผลลัพธ์** { nextState, reason } **เพื่อให้การเปลี่ยนสถานะมีความสอดคล้องและทดสอบง่าย.**

17.2 Middleware Access Matrix

Route group	Access condition	Behavior
---	---	---
Public pages	/, /pricing, /login, /partner/join, /api/public/*	No session required
Widget APIs	/api/widget/*, /api/embed/*, /api/public/*	Public access with domain/embed-key validation

Webhooks	/api/webhooks/*, /api/stripe/connect/callback	Validated by signature/token
Cron endpoints	/api/cron/*	Require INTERNAL_CRON_SECRET
Authenticated routes	/dashboard/*, /partner/*, /admin/*	Require valid JWT and role checks
Pending registration	role = pending	Allow onboarding/auth-only routes
Soft-delete account	pendingDeleteAt set	Allow recover and notice page only

17.3 Environment Variable Matrix

Variable	Required	Purpose
---	---	---
MONGO_URI	Yes	MongoDB connection string
AUTH_SECRET	Yes	NextAuth secret
AUTH_URL	Yes	Base auth URL
GOOGLE_CLIENT_ID	Yes	Google OAuth client id
GOOGLE_CLIENT_SECRET	Yes	Google OAuth client secret
GEMINI_API_KEY / GEMINI_API_KEY_LIVE	Yes (one of them)	Gemini AI access
INTERNAL_CRON_SECRET	Optional	Cron job authentication
STRIPE_SECRET_KEY / STRIPE_WEBHOOK_SECRET	Optional but required for billing	Subscription/webhook processing
MAIL_PROVIDER / MAIL_API_KEY / MAIL_FROM	Optional	Outbound email
ORDER_INGEST_SECRET / ZUORDER_INGEST_URL	Optional	Order integration relay

17.4 Monitoring, Logging, and Maintenance

- Health endpoint: /api/health
- Cron endpoints under /api/cron/* for maintenance and background jobs
- AI errors are parsed and normalized by apps/web/lib/ai/geminiErrors.ts
- RAG activity is logged through apps/web/lib/knowledge/ragEventLogger.ts
- Monitoring should include webhook failures, Gemini quota exhaustion, MongoDB connectivity, and cron job execution results

17.5 Data Retention and Privacy Controls

- PII scrubbing is applied in widget flows via apps/web/lib/security/unrecordPii.ts
- Tenant account deletion supports soft-delete and recovery before hard delete
- GDPR export/delete endpoints are available for tenant data handling
- Sensitive runtime secrets must never be exposed in UI, logs, or client responses

17.6 Integration Matrix

Integration	Purpose	Main entry point
---	---	---
Google OAuth	Authentication	apps/web/app/api/auth/[...nextauth]/route.ts
Gemini AI	Chat and embeddings	apps/web/lib/ai/*
Stripe	Billing and webhooks	apps/web/app/api/stripe/*
LINE / Meta / TikTok	Omnichannel inbound	apps/web/app/api/webhooks/*
Resend	Email delivery	apps/web/lib/services/*
Redis / Upstash	Rate limit and caching	apps/web/lib/security/rateLimit.ts

17.7 Sequence Diagram — Widget Chat Flow

```
Customer Browser -> apps/web/api/widget/init : POST embed key + origin
apps/web/api/widget/init -> MongoDB Atlas : validate tenant profile + allowed domains
apps/web/api/widget/init -> Customer Browser : return bot config

Customer Browser -> apps/web/api/widget/chat : POST message + sessionId
apps/web/api/widget/chat -> MongoDB Atlas : validate quota, session, tenant profile
apps/web/api/widget/chat -> Gemini AI : request response with context
Gemini AI -> apps/web/api/widget/chat : AI reply + structured output
apps/web/api/widget/chat -> MongoDB Atlas : persist conversation + usage + order draft
apps/web/api/widget/chat -> Customer Browser : return reply / products / handoff status
```

17.8 Sequence Diagram — Stripe Checkout and Webhook

```
Tenant/User -> apps/web/api/stripe/checkout : create checkout session
apps/web/api/stripe/checkout -> Stripe : POST checkout session
Stripe -> Tenant/User : redirect to payment page
Stripe -> apps/web/api/stripe/webhook : payment/subscription event
apps/web/api/stripe/webhook -> MongoDB Atlas : update Subscription + Invoice state
apps/web/api/stripe/webhook -> apps/web/lib/payment/botStateMachine : recalculate bot state
```

17.9 Deployment and Infrastructure Specification

Component	Specification	Notes
---	---	---
Hosting	AWS Amplify	Multi-app deployment for web/api/dashboard/landing
Runtime	Node.js + Next.js	SSR/API routes and server-side logic

Database	MongoDB Atlas	Main operational data store for all tenant/accounting/chat state
AI	Google Gemini	Chat and embedding services
Payment	Stripe	Checkout, subscription, webhook, Connect for partners
Messaging	LINE / Meta / TikTok webhooks	Omni-channel inbound and handoff
Email	Resend	Transactional notifications
Security	AWS Amplify env guard + middleware	No fallback secrets, no app-local runtime env usage

17.10 Version History

Version	Date	Summary
---	---	---
v1.0	2026-06-22	Initial TSD baseline from earlier technical specification
v1.1	2026-07-05	Added source-code aligned architecture, process flows, API spec, middleware mapping, error handling, and operational notes

---

18. Implementation Notes and Recommended Next Steps

1. Keep all production env vars in Amplify Console.

2. Maintain MongoDB Atlas connectivity and index health.

3. Monitor Gemini usage and quota events.

4. Validate Stripe webhook and cron secret handling on every environment.

5. Keep the widget bundle rebuilt whenever packages/widget changes.

6. Review bot state transitions after plan changes and quota exhaustion events.

7. Add CloudWatch/alerting for webhook and AI failure spikes.

---

19. Summary

ระบบ Zudobot เป็น SaaS platform สำหรับแชทบอทอัจฉริยะที่ผนวก Next.js, MongoDB Atlas, Gemini AI, Stripe, และ omnichannel webhook เข้าด้วยกัน โดยใช้แนวทาง modular architecture, middleware-based access control, และ pure-state machine สำหรับการจัดการสถานะบอทและโควต้า

เพื่อสนับสนุน tenant, partner, admin, และ widget-based customer interaction ในรูปแบบที่สามารถขยายได้และป้องกันความเสี่ยงจากการรั่วของ secret และการใช้ AI ที่ไม่ควบคุมได้